BankMigrate

Enterprise Banking Data Migration Platform

A complete, build-ready technical specification: architecture, database schemas, validation rules, T-SQL procedures, Python migration engine, ASP.NET Core API, scheduling, failure simulation, security, documentation and test plan.

Primary tech stack	Python · Pandas · SQLAlchemy · SQL Server (T-SQL) · ASP.NET Core (C#)
Project type	Simulated legacy-to-target banking data migration pipeline
Data used	100% synthetic — no real customer or banking data
Document purpose	Step-by-step build instructions, aligned to target resume bullets

Prepared as a build brief — read section by section, build in the order given.

Table of Contents

0. How to use this document

This document is a complete build specification for a project called **BankMigrate**. It is written to be read and executed step by step by an engineering build agent (for example, an AI coding assistant such as Antigravity) or by a developer building the project manually.

- Build the sections **in the order listed** in the Table of Contents — each later section depends on the one before it (e.g. the validation engine needs the legacy database to already exist).
- Every section states exactly **what to build**, **which technology to use**, and **what "done" looks like** for that piece.
- Section 24 maps every resume bullet back to the exact feature that must exist to justify it. **Do not add a claim to the resume until the corresponding feature is actually built and demonstrable.**
- Section 23 lists things that are explicitly **out of scope** — do not add them, even if they seem impressive. They dilute the core migration-engineering story.
- All data in this project is **synthetic**. No real customer, account, or transaction data should ever be used.

1. Project definition

BankMigrate is a Python, SQL Server/T-SQL, and ASP.NET Core platform that simulates an enterprise banking data migration. It extracts inconsistent, "dirty" legacy banking data, profiles and validates it, transforms it into a clean normalized target schema, loads the valid records, isolates the invalid records as tracked exceptions, reconciles the source and target systems record by record and dollar by dollar, and maintains automated migration run tracking, audit logging, scheduling, and failure-recovery workflows.

Core problem being simulated

A fictional bank has customer, account, loan, and transaction data sitting in an old legacy database. That data has duplicates, inconsistent formats, missing values, and invalid relationships. BankMigrate is the engineering system that moves that data safely into a new, clean, normalized banking system — without silently losing or corrupting anything.

2. Why this project exists — resume alignment

This project exists to genuinely demonstrate the skills required by a specific target job description (referred to below as "Q2"). Q2 requires hands-on evidence of:

- Migrating data from a legacy system into a target format
- Building automated / custom data processing
- Troubleshooting and resolving data-quality issues
- Production-style operational tracking (runs, exceptions, audit trails)
- SQL Server / T-SQL
- Python
- .NET
- Disciplined handling of security, availability, confidentiality, and privacy

*Note: The governing rule for this whole project: **build the feature first, then add the resume bullet.** The resume is a description of what was actually built — never the other way around. Section 24 gives the exact bullet-to-feature checklist to enforce this.*

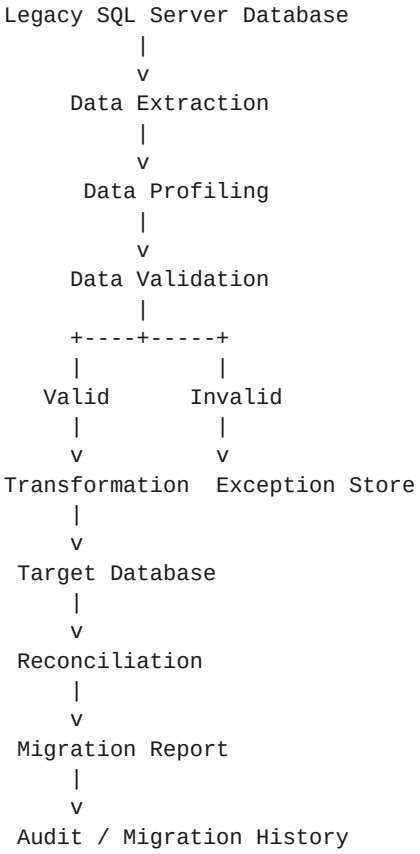
3. Full technology stack

Layer	Technology	Role in the project
Legacy & target database	Microsoft SQL Server	Source of truth for both the dirty legacy data and the clean target schema. Also hosts all stored procedures.
SQL scripting	T-SQL	Stored procedures, CTEs, window functions, temp tables, TRY...CATCH, transactions, constraints, indexes.
Migration engine	Python 3.x	Orchestrates extract → profile → validate → transform → load → reconcile → report.
Data handling	Pandas	Data profiling, cleaning, and transformation in memory.
DB connectivity (Python)	SQLAlchemy + pyodbc	Connects the Python engine to SQL Server.
Orchestration / API layer	.NET 8 — ASP.NET Core Web API (C#)	Exposes migration runs, exceptions, and reconciliation reports as REST endpoints; triggers the Python engine.
Scheduling	Windows Task Scheduler or a Python scheduler (e.g. APScheduler)	Runs the migration job automatically on a recurring schedule.
Testing	pytest (Python side), xUnit/NUnit (API side)	Automated tests for validation, transformation, reconciliation, and API endpoints.
Secrets & config	.env files + environment variables	Keeps credentials out of source control.
Version control	Git	Standard source control; .env excluded via .gitignore.

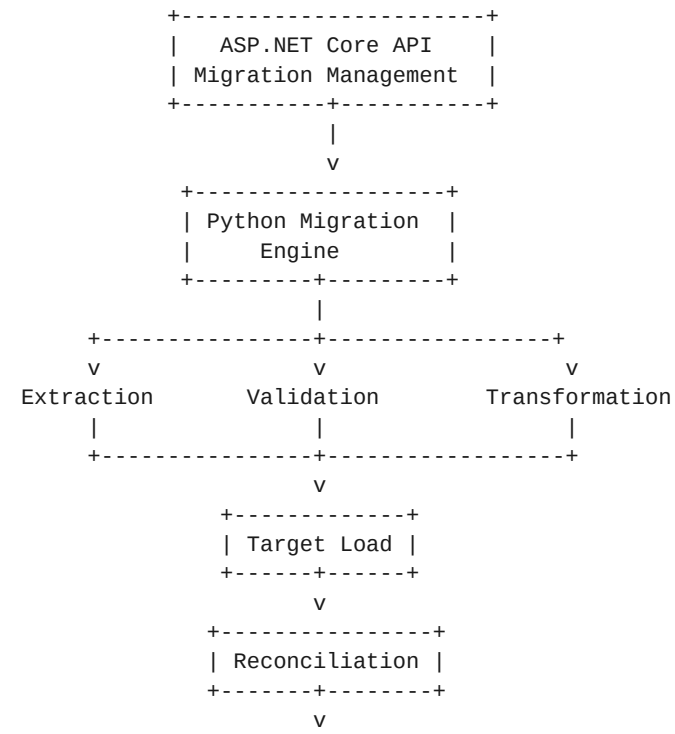
Note: This is the complete stack. Nothing else is required to satisfy the target job description — see Section 23 for things deliberately left out.

4. System architecture

4.1 End-to-end data flow



4.2 Component / service architecture



```
                    +----------------+
                    | Audit / Reports|
                    +----------------+

SQL Server
+-- Legacy Database  (BankMigrate_Legacy)
+-- Target Database  (BankMigrate_Target)
```

Why the .NET API exists at all: the ASP.NET Core API is the orchestration and reporting layer (it starts migration runs, and exposes run status, exceptions, and reconciliation results as REST endpoints). The Python engine is the actual data-processing worker. Do not build a pointless Python → .NET → SQL Server chain just to list both technologies — the API must have a real orchestration/reporting job, described in Section 17.

5. Build roadmap — order of implementation

Build the project in this exact order. Each step is a checkpoint — do not move to the next step until the current one runs and produces real output.

#	Milestone	What "done" looks like
1	Provision SQL Server	Install/connect to SQL Server. Create two databases: the legacy source and the clean target (Section 6 & 8).
2	Create and seed the legacy schema	Create the 6 legacy tables and generate synthetic data, deliberately including the data-quality problems in Section 7.
3	Create the target schema	Create the normalized target tables plus MigrationRuns, MigrationExceptions, MigrationAudit (Section 8).
4	Write the data mapping document	Formalize legacy-field to target-field mappings and transformation rules (Section 9).
5	Build the Python migration engine skeleton	Set up the module structure: extraction, profiling, validation, transformation, loading, reconciliation, exceptions, audit, config (Section 10).
6	Implement extraction + profiling	Pull legacy data into pandas DataFrames; produce basic profiling stats (row counts, null counts, duplicate counts).
7	Implement the validation engine	Apply the rule catalog in Section 11 and produce pass/fail results per record.
8	Implement transformation	Apply the mapping + cleaning rules from Section 9 to valid records only.
9	Implement loading	Insert transformed, valid records into the target database.
10	Build the T-SQL stored procedure layer	Implement the procedures in Section 12 directly in SQL Server; call the relevant ones from Python where appropriate.
11	Implement exception handling	Write rejected records into MigrationExceptions with rule id, severity, and message (Section 13).
12	Implement reconciliation	Compare source vs. valid vs. loaded vs. rejected counts and amounts (Section 14).
13	Implement run tracking + audit logging	Wrap every run in a MigrationRuns record; log every insert/update/reject to MigrationAudit (Sections 15 & 16).
14	Build the ASP.NET Core API	Expose the endpoints in Section 17, calling into the Python engine and/or SQL Server directly for reporting.
15	Add scheduling	Wire up a recurring automated run using Task Scheduler or a Python scheduler (Section 18).
16	Implement failure simulation & recovery	Deliberately trigger the 4 failure scenarios in Section 19 and confirm the system logs, isolates, and can resume safely.
17	Apply security hardening	Work through the Section 20 checklist (env vars, parameterized queries, least privilege, no real data).
18	Write documentation	Produce every file listed in Section 21.
19	Write automated tests	Cover the test files listed in Section 22 and get them passing.
20	Finalize resume claims	Go through Section 24 line by line and only keep bullets that are fully demonstrable.

6. Legacy source database (SQL Server)

Database name: **BankMigrate_Legacy**. Exactly six tables — do not over-build this.

```
BankMigrate_Legacy
+-- Customers_Legacy
+-- Accounts_Legacy
+-- Transactions_Legacy
+-- Loans_Legacy
+-- Beneficiaries_Legacy
+-- Addresses_Legacy
```

Customers_Legacy — example columns

Column	Notes
customer_id	Primary key; some rows deliberately NULL (Section 7)
customer_name	Free-text, inconsistent casing/spacing
dob	Free-text date, inconsistent formats
phone	Inconsistent formats (+91-, spaces, no country code)
email	Inconsistent casing, stray whitespace
address_id	Foreign key to Addresses_Legacy

Accounts_Legacy — example columns

Column	Notes
account_id	Primary key
customer_id	Foreign key to Customers_Legacy (some invalid on purpose)
account_type	e.g. SAVINGS, CURRENT, LOAN
balance	Some invalid/negative-where-not-allowed values
opened_date	Inconsistent date formats
status	ACTIVE, CLOSED, DORMANT

Transactions_Legacy, Loans_Legacy, Beneficiaries_Legacy, Addresses_Legacy

Build these with the same philosophy: minimal realistic columns for a banking record, with transaction_id, account_id (FK), transaction_type, amount, transaction_date, description for Transactions_Legacy, and analogous natural fields for the remaining three tables. Keep the schema small and purposeful — six tables total is enough to prove the migration pattern.

7. Intentional data-quality problems

The legacy data must be deliberately dirty. If the source data is already clean, there is nothing real for the migration engine to solve. Seed the following problem types into the legacy database:

Problem type	Example
Duplicate customers	C001 'John Smith' and C019 'JOHN SMITH' representing the same person
Email formatting inconsistencies	' john.smith@gmail.com ' vs 'JOHN.SMITH@GMAIL.COM'
Phone inconsistencies	+91-9876543210 / 09876543210 / '98765 43210'
Invalid dates	31/02/1999 (day does not exist)
Missing mandatory values	customer_id = NULL
Invalid foreign keys	transaction.account_id references A999999, which does not exist
Duplicate transactions	The exact same transaction row appears twice
Invalid balances	Balances that violate whatever business rule you define (e.g. negative savings balance)

Note: Seed a small, deliberate, documented set of these problems (not random corruption) so every issue is traceable back to a known validation rule in Section 11.

8. Target database (SQL Server)

Database name: **BankMigrate_Target**. This schema is clean and normalized — it is not a copy of the legacy schema.

```
BankMigrate_Target
+-- Customers
+-- Accounts
+-- Transactions
+-- Loans
+-- Beneficiaries
+-- Addresses
+-- MigrationRuns
+-- MigrationExceptions
+-- MigrationAudit
```

Key concept: legacy schema ≠ target schema. This is not a copy operation. The pipeline must perform real schema mapping, transformation, and validation before loading — that is what makes this a migration project rather than a data copy script.

9. Data mapping — legacy to target

Produce a formal, version-controlled mapping document (see Section 21) with entries like:

Legacy field	Target field
cust_id	customer_id
cust_name	full_name
birth_date	date_of_birth
mobile_no	phone_number
acct_no	account_number
acct_type	account_type
txn_amt	transaction_amount
txn_dt	transaction_date

Transformation rule examples

```
'  JOHN  SMITH  '        -->  'John Smith'
'+91-98765-43210'        -->  '9876543210'
'12/03/1999'             -->  1999-03-12  (ISO date)
```

Note: This mapping + transformation layer is what supports the resume phrase "legacy-to-target banking data migration pipeline." It must exist and be documented, not implied.

10. Python migration engine

Core libraries: **Python, Pandas, SQLAlchemy, pyodbc**. Structure the project as a proper package, not a single script:

```
migration_engine/
+-- extraction/
+-- profiling/
+-- validation/
+-- transformation/
+-- loading/
+-- reconciliation/
+-- exceptions/
+-- audit/
+-- config/
```

The engine orchestrates one linear pipeline per run:

```
extract()
    |
    v
profile()
    |
    v
validate()
    |
    v
transform()
    |
    v
load()
    |
    v
reconcile()
    |
    v
report()
```

Note: This module is what supports the resume phrase "Python migration services." Each function above should be a real, independently callable, independently testable unit.

11. Validation engine and rules

Every rule needs a stable rule ID, so failures are traceable and reportable. Minimum rule catalog:

Customer rules

Rule ID	Meaning
CUSTOMER_001	Customer ID required
CUSTOMER_002	Duplicate customer
CUSTOMER_003	Invalid phone
CUSTOMER_004	Invalid email
CUSTOMER_005	Invalid date of birth

Account rules

Rule ID	Meaning
ACCOUNT_001	Account ID required
ACCOUNT_002	Referenced customer must exist
ACCOUNT_003	Valid account type
ACCOUNT_004	Valid balance

Transaction rules

Rule ID	Meaning
TXN_001	Transaction ID required
TXN_002	Referenced account must exist
TXN_003	Valid amount
TXN_004	Valid transaction date
TXN_005	Duplicate transaction

Example validation output

```
Record:   TXN-89231
Rule:     TXN_002
Severity: ERROR
Message:  Referenced account does not exist
```

12. T-SQL layer — stored procedures

This layer specifically demonstrates advanced SQL Server / T-SQL ability. Writing only SELECT statements is not sufficient. Implement, at minimum, these stored procedures:

- **sp_validate_customers** — runs the CUSTOMER_* rule set against staged legacy data
- **sp_validate_accounts** — runs the ACCOUNT_* rule set
- **sp_validate_transactions** — runs the TXN_* rule set
- **sp_detect_duplicates** — window-function-based duplicate detection across customers/transactions
- **sp_reconcile_migration** — compares source vs. target counts and amounts for a given run
- **sp_generate_migration_summary** — produces the run-level summary report

Required T-SQL techniques to demonstrate across these procedures:

- CTEs (Common Table Expressions)
- Window functions (e.g. ROW_NUMBER, PARTITION BY for duplicate detection)
- Temporary tables / table variables
- Stored procedures with parameters
- Explicit transactions (BEGIN TRAN / COMMIT / ROLLBACK)
- TRY...CATCH error handling
- Indexes and constraints on both schemas
- Aggregation and JOIN-heavy reconciliation queries

Note: The goal is not to show off SQL syntax for its own sake — it is to prove SQL Server is a real working part of the migration system, not decoration.

13. Exception handling

Invalid data must never simply disappear. Every rejected record is written to the MigrationExceptions table.

Column	Purpose
exception_id	Primary key
run_id	Which migration run produced this exception
entity_type	Customer / Account / Transaction / Loan / etc.
record_id	The natural ID of the failing record
rule_id	Which validation rule failed (e.g. TXN_002)
severity	ERROR / WARNING
error_message	Human-readable explanation
source_data	Snapshot of the original offending record
created_at	Timestamp
status	OPEN / RESOLVED / IGNORED

Example row

```
EX-10291 | Transaction | TXN-89231 | TXN_002 | ERROR
Message: Account A999 does not exist
Status:  OPEN
```

Note: This is the feature that lets you honestly claim: "the migration engine isolates rejected records rather than silently dropping them."

14. Reconciliation

For every run, the system must automatically prove that no records were silently lost.

```
Source customers:        10,000
Valid customers:           9,982
Rejected customers:           18
Loaded customers:          9,982

Check:  Source = Valid + Rejected   -->  10,000 = 9,982 + 18  (OK)
```

Apply the same pattern to accounts and transactions, and additionally reconcile monetary totals:

```
Source transaction amount
     =
Target transaction amount + Rejected transaction amount
   (with any exclusions explicitly documented)
```

Note: This is the feature that justifies the word "reconciliation" on the resume — it must run automatically at the end of every migration, not be a manual spot-check.

15. Migration run tracking

Every execution of the pipeline gets a unique run ID, e.g. `RUN-20260821-001`. Track, per run:

- started_at / completed_at
- source_records
- validated_records
- transformed_records
- loaded_records
- rejected_records
- status

Example run summary

```
RUN-001
Customers   Source 10,000   Validated 9,982   Rejected 18   Loaded 9,982
Accounts    Source  8,500   Validated 8,471   Rejected 29   Loaded 8,471

Status: COMPLETED_WITH_EXCEPTIONS
```

Note: This is what gives the system operational visibility — the same shape of reporting a real production migration job would need.

16. Audit logging

Track every meaningful operation in MigrationAudit:

Column	Purpose
run_id	Which run this event belongs to
entity	Customer / Account / Transaction / etc.
record_id	Natural ID of the affected record
operation	INSERT / UPDATE / REJECT
timestamp	When it happened
status	SUCCESS / FAILURE

```
RUN-001 | Customer | C001 | INSERT | 2026-08-21 14:32:10 | SUCCESS
```

Note: You do not need full before/after snapshots for every record — keep this practical. This table is what justifies the resume phrase "automated migration audit logging."

17. ASP.NET Core API

This is where .NET becomes a real, legitimate part of the system — not just a resume line.

Method & route	Purpose
POST /api/migrations	Start a new migration run (triggers the Python engine)
GET /api/migrations	List past and current runs
GET /api/migrations/{runId}	Get status/details of one run
GET /api/migrations/{runId}/exceptions	List rejected records for a run
GET /api/migrations/{runId}/reconciliation	Get the reconciliation report for a run
POST /api/migrations/{runId}/retry	Retry a failed or partially completed run

Suggested internal structure

```
ASP.NET Core API
  +-- Migration Controller
  +-- Migration Service
  +-- Reporting Service
          |
          v
   Python Migration Engine
          |
          v
      SQL Server
```

Note: Do not build a needless Python -> .NET -> SQL Server chain purely to list both technologies. The API's real job is orchestration (starting/retrying runs) and reporting (exposing run status, exceptions, and reconciliation as REST resources) — that is a genuine, defensible reason for its existence, not decoration.

18. Scheduling

The target job description explicitly requires "construct and schedule automated processes." The pipeline must therefore run on a schedule, not only on demand.

```
02:00 AM
   |
   v
Migration job starts
   |
   v
Extract -> Validate -> Transform -> Load -> Reconcile -> Generate report
```

For development, use either:

- Windows Task Scheduler calling a Python entry-point script, or
- A Python scheduler library (e.g. APScheduler) running the same pipeline on a cron-style interval

Note: The exact scheduler technology matters less than proving the process is repeatable and automated, not manually triggered every time.

19. Failure simulation scenarios

Without deliberate failure scenarios, a "troubleshooting" claim on the resume is unsupported. Build and demonstrate all four:

STEP 1 Duplicate customer

System detects the duplicate, rejects the record, logs it to MigrationExceptions, and continues processing the rest of the batch without stopping.

STEP 2 Invalid foreign key

A transaction references a nonexistent account. Validation fails on rule TXN_002, and an exception record is created with the source data preserved.

STEP 3 Database connection failure

SQL Server becomes unavailable mid-run. The engine detects the connection error, retries with backoff, and logs the failure if retries are exhausted.

STEP 4 Partial migration

Example: 5,000 of 10,000 records are processed when the connection drops. The system must be able to answer: what was loaded, what wasn't, and can the run resume safely from where it stopped?

Note: This section is what turns the project from a college CRUD exercise into a system that looks like real production engineering.

20. Security requirements

Do not overclaim — this is not a real banking security platform. Implement a credible baseline:

- Database credentials stored in environment variables, never hard-coded
- No passwords or secrets ever committed to Git
- `.env` file excluded via `.gitignore`
- All SQL access uses parameterized queries (no string-concatenated SQL)
- A least-privilege database user/service account for the migration engine
- Input validation on every API endpoint
- Audit logging of data-changing operations (Section 16)
- Only synthetic, clearly-labeled test data — never real customer information

Note: Explicitly document in the repo: "All banking data used by this project is synthetic." This is the credible, honest answer to the security/privacy requirement in the target job description.

21. Documentation to produce

This project is not complete without these files, all under a top-level `docs/` folder:

```
docs/
+-- architecture.md
+-- data-mapping.md
+-- validation-rules.md
+-- migration-runbook.md
+-- troubleshooting.md
+-- change-control.md
+-- security.md
```

migration-runbook.md — required contents

```
1. Verify source database
2. Validate connectivity
3. Create migration run
4. Execute extraction
5. Run validation
6. Review exceptions
7. Transform valid records
8. Load target
9. Run reconciliation
10. Review migration report
```

troubleshooting.md — example entry format

```
Problem: Foreign-key violations

Possible causes:
- Missing parent records
- Incorrect mapping
- Source data corruption

Resolution:
- Validate source references
- Review mapping configuration
- Re-run affected records
```

22. Testing requirements

Minimum automated test suite:

```
tests/
+-- test_validation.py
+-- test_transformation.py
+-- test_reconciliation.py
+-- test_migration.py
+-- test_api.py
```

Each test file should cover, at minimum:

- Valid records pass through unchanged
- Invalid records are correctly rejected with the right rule ID
- Duplicate detection (customers and transactions)
- Null / missing mandatory values
- Malformed dates
- Invalid foreign key references
- Transformation rules (name casing, phone normalization, date parsing)
- Reconciliation math (source = valid + rejected, and monetary totals)
- API endpoint responses (status codes and payload shape)

23. Explicitly out of scope

Do not add any of the following. They dilute the core migration-engineering story and are not required by the target job description:

React dashboard
Kubernetes
Kafka
AWS architecture
Microservices sprawl
Terraform
20+ database tables
Real banking APIs
Actual bank data
Blockchain
Machine learning
A polished frontend
A mobile application

Priority: data migration engineering first. Everything in this project should serve that story.

24. Resume bullet to feature traceability

Only keep a resume bullet if every feature it depends on is actually built and demonstrable. Use this table as the final checklist before publishing the resume.

Bullet 1

"Engineered an end-to-end legacy-to-target banking data migration pipeline covering extraction, validation, transformation, loading, and reconciliation."

- Legacy database exists (Section 6)
- Target database exists (Section 8)
- Extraction implemented (Section 10)
- Validation implemented (Section 11)
- Transformation implemented (Sections 9 & 10)
- Loading implemented (Section 10)
- Reconciliation implemented (Section 14)
- Customer / account / loan / transaction entities all present

Bullet 2

"Developed Python migration services and T-SQL validation procedures for duplicate detection, referential integrity, data-quality checks, exception handling, and automated migration audit logging."

- Python migration modules exist (Section 10)
- T-SQL stored procedures exist (Section 12)
- Duplicate detection implemented (sp_detect_duplicates / CUSTOMER_002 / TXN_005)
- Foreign-key / referential integrity checks implemented (ACCOUNT_002 / TXN_002)
- Data-quality rule catalog implemented (Section 11)
- MigrationExceptions table implemented (Section 13)
- MigrationAudit table implemented (Section 16)

Bullet 3

"Built an ASP.NET Core REST API and scheduled migration workflow with failure simulation, reconciliation reporting, change tracking, and recovery-oriented troubleshooting."

- ASP.NET Core API implemented with the endpoints in Section 17
- Scheduled job implemented (Section 18)
- All four failure scenarios implemented and demonstrable (Section 19)
- Reconciliation report exposed via the API (Section 17, Section 14)
- Change tracking = MigrationAudit (Section 16)
- Troubleshooting/recovery = documented in troubleshooting.md and proven in Scenario 4 (Section 19)

Note: Rule: if any single checklist item under a bullet is missing, remove or reword that resume bullet until it is true.

25. Definition of done

The project is considered complete and “resume-ready” only when all of the following are true:

- Both databases exist in SQL Server with the schemas defined in Sections 6 and 8
- The legacy database contains deliberately seeded, documented data-quality problems (Section 7)
- The Python migration engine runs the full extract → profile → validate → transform → load → reconcile → report pipeline end to end
- All six T-SQL stored procedures exist and are actually called as part of the pipeline (Section 12)
- Rejected records are visible in MigrationExceptions with correct rule IDs (Section 13)
- A reconciliation report proves source = valid + rejected for every run (Section 14)
- Every run is tracked with a unique run ID and full record counts (Section 15)
- MigrationAudit contains a row for every meaningful operation (Section 16)
- The ASP.NET Core API exposes and correctly serves all six endpoints (Section 17)
- The pipeline runs automatically on a schedule, not only manually (Section 18)
- All four failure scenarios have been triggered at least once and behaved as specified (Section 19)
- The security checklist in Section 20 is fully applied
- All seven documentation files in Section 21 exist and are accurate
- All five test files in Section 22 exist and pass
- The final resume bullets have been checked line-by-line against Section 24 and are all true

---

End of specification. Build sections in order (Section 5), verify each milestone before continuing, and only finalize resume claims once Section 25 is fully satisfied.